

CircleUp — Day 5

Core Feature Development: Circles (Browse, Create, Join/Leave)

Every file below is complete and copy-pasteable.

All tools used are free — no paid API keys required anywhere in this guide.

Firestore Security Rules

What this is: Rules that control who can read/write your database. Your Firestore is likely still in "test mode" from Day 3, which is wide open and expires after 30 days. We lock it down properly here — still on Firebase's free Spark plan, no payment involved.

Manual steps (Firebase Console)

1. Go to <https://console.firebase.google.com> and open your CircleUp project.
2. Left sidebar → **Build** → **Firestore Database**.
3. Click the **Rules** tab at the top.
4. Select all existing text in the editor and delete it.
5. Paste in the rules below.
6. Click the blue **Publish** button (top right).
7. Confirm you see "Rules published successfully."

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /users/{userId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null && request.auth.uid == userId;
    }

    match /circles/{circleId} {
      allow read: if request.auth != null;
      allow create: if request.auth != null;
      allow update: if request.auth != null;
      allow delete: if false;
    }
  }
}
```

Plain English: Any logged-in user can read circles and their own profile; only you can edit your own user document; anyone logged in can create/join circles; nobody can delete a circle through the app (keeps data safe while building).

Circle Service (Data Layer)

What we're building: A single file with all Firestore functions for circles — create, list (live-updating), get one, join, leave. Every screen in this app calls these functions instead of touching Firestore directly, so logic stays in one place.

Open Command Prompt in your project folder and run:

```
npx expo install @react-navigation/bottom-tabs
```

This adds free, open-source bottom tab navigation (Browse / My Circles / Create / Profile). No paid keys involved — `@expo/vector-icons` (used for tab icons) ships with Expo by default at no cost.

File 1 — circleService.js NEW

```
src/services/circleService.js
```

```

import {
  collection,
  addDoc,
  getDoc,
  doc,
  updateDoc,
  arrayUnion,
  arrayRemove,
  onSnapshot,
  serverTimestamp,
} from "firebase/firestore";
import { db } from "../firebase";

const circlesRef = collection(db, "circles");

// Creates a new circle with the current user as its first member.
export async function createCircle(name, description, creatorUid) {
  const docRef = await addDoc(circlesRef, {
    name: name.trim(),
    description: description.trim(),
    memberIds: [creatorUid],
    createdBy: creatorUid,
    createdAt: serverTimestamp(),
  });
  return docRef.id;
}

// Live-subscribes to ALL circles. Calls callback(circles[]) every time
// the data changes (new circle created, someone joins, etc.).
export function subscribeToAllCircles(callback) {
  return onSnapshot(circlesRef, (snapshot) => {
    const circles = snapshot.docs.map((d) => ({ id: d.id, ...d.data() }));
    circles.sort(
      (a, b) => (b.createdAt?.seconds || 0) - (a.createdAt?.seconds || 0)
    );
    callback(circles);
  });
}

// Fetches a single circle by its ID (used on the Circle Detail screen).
export async function getCircleById(circleId) {
  const snap = await getDoc(doc(db, "circles", circleId));
  if (!snap.exists()) return null;
  return { id: snap.id, ...snap.data() };
}

// Adds the given user's ID to a circle's member list.
export async function joinCircle(circleId, uid) {
  await updateDoc(doc(db, "circles", circleId), {
    memberIds: arrayUnion(uid),
  });
}

// Removes the given user's ID from a circle's member list.
export async function leaveCircle(circleId, uid) {
  await updateDoc(doc(db, "circles", circleId), {
    memberIds: arrayRemove(uid),
  });
}

```

Where it goes: `src/services/circleService.js` — brand new file. Imports `db` from your existing `src/services/firebase.js` (Day 3) — do not modify that file.

File 2 — Input.js MODIFIED

`src/components/Input.js` — replaces the Day 4 version

```
import React from "react";
import { TextInput, StyleSheet, Text, View } from "react-native";

export default function Input({ label, error, style, ...props }) {
  return (
    <View style={styles.wrapper}>
      {label ? <Text style={styles.label}>{label}</Text> : null}
      <TextInput
        style={[styles.input, error ? styles.inputError : null, style]}
        placeholderTextColor="#9ca3af"
        autoCapitalize="none"
        {...props}
      />
      {error ? <Text style={styles.error}>{error}</Text> : null}
    </View>
  );
}

const styles = StyleSheet.create({
  wrapper: { marginBottom: 16 },
  label: { fontSize: 14, fontWeight: "600", marginBottom: 6, color: "#374151" },
  input: {
    borderWidth: 1,
    borderColor: "#d1d5db",
    borderRadius: 10,
    paddingHorizontal: 14,
    paddingVertical: 12,
    fontSize: 16,
    backgroundColor: "fff",
  },
  inputError: { borderColor: "#ef4444" },
  error: { color: "#ef4444", fontSize: 12, marginTop: 4 },
});
```

Why this changed: The Create Circle screen needs a taller, multi-line description field. This adds an optional `style` prop so any screen can customize the box (e.g. height) without breaking the default look used by Login/Signup.

DAY 5 · MILESTONE 3

Browse & Create Circle Screens

What we're building: A reusable `CircleCard` component, the Browse Circles screen (live list of all circles), and the Create Circle screen (form that writes a new circle to Firestore).

File 1 — CircleCard.js NEW

src/components/CircleCard.js

```
import React from "react";
import { View, Text, StyleSheet, TouchableOpacity } from "react-native";

export default function CircleCard({ circle, onPress, isMember }) {
  const memberCount = circle.memberIds?.length || 0;

  return (
    <TouchableOpacity style={styles.card} onPress={onPress} activeOpacity={0.8}>
      <View style={styles.header}>
        <Text style={styles.name}>{circle.name}</Text>
        {isMember ? (
          <View style={styles.badge}>
            <Text style={styles.badgeText}>Joined</Text>
          </View>
        ) : null}
      </View>
      <Text style={styles.description} numberOfLines={2}>
        {circle.description}
      </Text>
      <Text style={styles.members}>
        {memberCount} member{memberCount === 1 ? "" : "s"}
      </Text>
    </TouchableOpacity>
  );
}

const styles = StyleSheet.create({
  card: { backgroundColor: "#fff", borderRadius: 12, padding: 16, marginBottom: 12,
borderWidth: 1, borderColor: "#e5e7eb" },
  header: { flexDirection: "row", justifyContent: "space-between", alignItems: "center",
marginBottom: 6 },
  name: { fontSize: 17, fontWeight: "700", color: "#111827" },
  badge: { backgroundColor: "#dcfce7", borderRadius: 8, paddingHorizontal: 8,
paddingVertical: 3 },
  badgeText: { fontSize: 11, fontWeight: "700", color: "#166534" },
  description: { fontSize: 14, color: "#6b7280", marginBottom: 8 },
  members: { fontSize: 12, color: "#9ca3af", fontWeight: "600" },
});
```

Where it goes: `src/components/CircleCard.js` — brand new file.

File 2 — BrowseCirclesScreen.js NEW

src/screens/BrowseCirclesScreen.js

```

import React, { useEffect, useState } from "react";
import { View, FlatList, Text, StyleSheet, ActivityIndicator } from "react-native";
import CircleCard from "../components/CircleCard";
import { subscribeToAllCircles } from "../services/circleService";
import { useAuth } from "../context/AuthContext";

export default function BrowseCirclesScreen({ navigation }) {
  const { user } = useAuth();
  const [circles, setCircles] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const unsubscribe = subscribeToAllCircles((data) => {
      setCircles(data);
      setLoading(false);
    });
    return unsubscribe;
  }, []);

  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#2563eb" />
      </View>
    );
  }

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Browse Circles</Text>
      <FlatList
        data={circles}
        keyExtractor={({id}) => id}
        contentContainerStyle={{ paddingBottom: 20 }}
        renderItem={({ item }) => (
          <CircleCard
            circle={item}
            isMember={item.memberIds?.includes(user.uid)}
            onPress={() => navigation.navigate("CircleDetail", { circleId: item.id })}
          />
        )}
        ListEmptyComponent={
          <Text style={styles.empty}>No circles yet. Be the first to create one!</Text>
        }
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#fafafa", padding: 16 },
  center: { flex: 1, justifyContent: "center", alignItems: "center" },
  title: { fontSize: 24, fontWeight: "800", color: "#111827", marginBottom: 16 },
  empty: { textAlign: "center", color: "#9ca3af", marginTop: 40, fontSize: 14 },
});

```

Where it goes: `src/screens/BrowseCirclesScreen.js` — brand new file.

File 3 — CreateCircleScreen.js NEW

src/screens/CreateCircleScreen.js

```

import React, { useState } from "react";
import { View, Text, StyleSheet, ScrollView, KeyboardAvoidingView, Platform } from "react-native";
import Input from "../components/Input";
import Button from "../components/Button";
import { createCircle } from "../services/circleService";
import { useAuth } from "../context/AuthContext";

export default function CreateCircleScreen({ navigation }) {
  const { user } = useAuth();
  const [name, setName] = useState("");
  const [description, setDescription] = useState("");
  const [loading, setLoading] = useState(false);
  const [errorMsg, setErrorMsg] = useState("");

  const handleCreate = async () => {
    setErrorMsg("");
    if (!name.trim() || !description.trim()) {
      setErrorMsg("Please fill in both fields.");
      return;
    }

    setLoading(true);
    try {
      const circleId = await createCircle(name, description, user.uid);
      setName("");
      setDescription("");
      navigation.navigate("BrowseTab", {
        screen: "CircleDetail",
        params: { circleId },
      });
    } catch (error) {
      setErrorMsg("Couldn't create the circle. Please try again.");
    } finally {
      setLoading(false);
    }
  };

  return (
    <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === "ios" ? "padding" : undefined}>
      <ScrollView contentContainerStyle={styles.container}>
        <Text style={styles.title}>Create a Circle</Text>
        <Text style={styles.subtitle}>Start a space for people who share your interest.</Text>

        <Input label="Circle Name" placeholder="e.g. Weekend Hikers" value={name}
onChangeText={setName} />
        <Input
          label="Description"
          placeholder="What's this circle about?"
          value={description}
          onChangeText={setDescription}
          multiline
          numberOfLines={4}
          style={{ height: 100, textAlignVertical: "top" }}
        />

        {errorMsg ? <Text style={styles.errorBanner}>{errorMsg}</Text> : null}
      </ScrollView>
    </KeyboardAvoidingView>
  );
}

```

```

        <Button title="Create Circle" onPress={handleCreate} loading={loading} />
      </ScrollView>
    </KeyboardAvoidingView>
  );
}

const styles = StyleSheet.create({
  container: { flexGrow: 1, padding: 24, backgroundColor: "#fafafa" },
  title: { fontSize: 26, fontWeight: "800", color: "#111827", marginBottom: 6 },
  subtitle: { fontSize: 14, color: "#6b7280", marginBottom: 28 },
  errorBanner: { backgroundColor: "#fef2f2", color: "#b91c1c", padding: 10, borderRadius:
8, marginBottom: 14, fontSize: 13 },
});

```

Where it goes: `src/screens/CreateCircleScreen.js` — brand new file.

DAY 5 · MILESTONE 4

My Circles, Circle Detail & Navigation Tabs

What we're building: The My Circles screen, the Circle Detail screen (with Join/Leave), and a bottom tab bar that replaces the single Home screen from Day 4 — this is the visual centerpiece of today's work.

File 1 — MyCirclesScreen.js NEW

`src/screens/MyCirclesScreen.js`

```

import React, { useEffect, useState } from "react";
import { View, FlatList, Text, StyleSheet, ActivityIndicator } from "react-native";
import CircleCard from "../components/CircleCard";
import { subscribeToAllCircles } from "../services/circleService";
import { useAuth } from "../context/AuthContext";

export default function MyCirclesScreen({ navigation }) {
  const { user } = useAuth();
  const [circles, setCircles] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const unsubscribe = subscribeToAllCircles((data) => {
      setCircles(data.filter((c) => c.memberIds?.includes(user.uid)));
      setLoading(false);
    });
    return unsubscribe;
  }, []);

  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#2563eb" />
      </View>
    );
  }

  return (
    <View style={styles.container}>
      <Text style={styles.title}>My Circles</Text>
      <FlatList
        data={circles}
        keyExtractor={({id}) => id}
        contentContainerStyle={{ paddingBottom: 20 }}
        renderItem={({ item }) => (
          <CircleCard
            circle={item}
            isMember={true}
            onPress={() => navigation.navigate("CircleDetail", { circleId: item.id })}
          />
        )}
        ListEmptyComponent={
          <Text style={styles.empty}>You haven't joined any circles yet. Browse to find
one!</Text>
        }
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#fafafa", padding: 16 },
  center: { flex: 1, justifyContent: "center", alignItems: "center" },
  title: { fontSize: 24, fontWeight: "800", color: "#111827", marginBottom: 16 },
  empty: { textAlign: "center", color: "#9ca3af", marginTop: 40, fontSize: 14 },
});

```

Where it goes: `src/screens/MyCirclesScreen.js` — brand new file.

File 2 — CircleDetailScreen.js NEW

src/screens/CircleDetailScreen.js

```

import React, { useEffect, useState } from "react";
import { View, Text, StyleSheet, ActivityIndicator, ScrollView } from "react-native";
import Button from "../components/Button";
import { getCircleById, joinCircle, leaveCircle } from "../services/circleService";
import { useAuth } from "../context/AuthContext";

export default function CircleDetailScreen({ route }) {
  const { circleId } = route.params;
  const { user } = useAuth();
  const [circle, setCircle] = useState(null);
  const [loading, setLoading] = useState(true);
  const [actionLoading, setActionLoading] = useState(false);

  const loadCircle = async () => {
    const data = await getCircleById(circleId);
    setCircle(data);
    setLoading(false);
  };

  useEffect(() => {
    loadCircle();
  }, [circleId]);

  const isMember = circle?.memberIds?.includes(user.uid);

  const handleJoin = async () => {
    setActionLoading(true);
    await joinCircle(circleId, user.uid);
    await loadCircle();
    setActionLoading(false);
  };

  const handleLeave = async () => {
    setActionLoading(true);
    await leaveCircle(circleId, user.uid);
    await loadCircle();
    setActionLoading(false);
  };

  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#2563eb" />
      </View>
    );
  }

  if (!circle) {
    return (
      <View style={styles.center}>
        <Text>Circle not found.</Text>
      </View>
    );
  }

  return (
    <ScrollView contentContainerStyle={styles.container}>
      <Text style={styles.name}>{circle.name}</Text>
      <Text style={styles.members}>
        {circle.memberIds?.length || 0} member{(circle.memberIds?.length || 0) === 1 ? "" :

```

```

"s"}
  </Text>
  <Text style={styles.description}>{circle.description}</Text>

  {isMember ? (
    <Button title="Leave Circle" onPress={handleLeave} loading={actionLoading}
variant="secondary" />
  ) : (
    <Button title="Join Circle" onPress={handleJoin} loading={actionLoading} />
  )}
</ScrollView>
);
}

const styles = StyleSheet.create({
  container: { padding: 24, backgroundColor: "#fafafa", flexGrow: 1 },
  center: { flex: 1, justifyContent: "center", alignItems: "center" },
  name: { fontSize: 26, fontWeight: "800", color: "#111827", marginBottom: 6 },
  members: { fontSize: 13, color: "#9ca3af", fontWeight: "600", marginBottom: 20 },
  description: { fontSize: 15, color: "#374151", lineHeight: 22, marginBottom: 30 },
});

```

Where it goes: `src/screens/CircleDetailScreen.js` — brand new file.

File 3 — BrowseStackNavigator.js NEW

`src/navigation/BrowseStackNavigator.js`

```

import React from "react";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import BrowseCirclesScreen from "../screens/BrowseCirclesScreen";
import CircleDetailScreen from "../screens/CircleDetailScreen";

const Stack = createNativeStackNavigator();

export default function BrowseStackNavigator() {
  return (
    <Stack.Navigator screenOptions={{ headerShown: false }}>
      <Stack.Screen name="BrowseCircles" component={BrowseCirclesScreen} />
      <Stack.Screen
        name="CircleDetail"
        component={CircleDetailScreen}
        options={{ headerShown: true, title: "" }}
      />
    </Stack.Navigator>
  );
}

```

Where it goes: `src/navigation/BrowseStackNavigator.js` — brand new file.

File 4 — MyCirclesStackNavigator.js NEW

src/navigation/MyCirclesStackNavigator.js

```
import React from "react";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import MyCirclesScreen from "../screens/MyCirclesScreen";
import CircleDetailScreen from "../screens/CircleDetailScreen";

const Stack = createNativeStackNavigator();

export default function MyCirclesStackNavigator() {
  return (
    <Stack.Navigator screenOptions={{ headerShown: false }}>
      <Stack.Screen name="MyCirclesList" component={MyCirclesScreen} />
      <Stack.Screen
        name="CircleDetail"
        component={CircleDetailScreen}
        options={{ headerShown: true, title: "" }}
      />
    </Stack.Navigator>
  );
}
```

Where it goes: src/navigation/MyCirclesStackNavigator.js — brand new file.

File 5 — HomeScreen.js MODIFIED

src/screens/HomeScreen.js — replaces the Day 4 version (now used as the Profile tab)


```
import React from "react";
import { View, Text, StyleSheet } from "react-native";
import Button from "../components/Button";
import { useAuth } from "../context/AuthContext";
import { logOut } from "../services/authService";

export default function HomeScreen() {
  const { user } = useAuth();

  return (
    <View style={styles.container}>
      <Text style={styles.title}>{user?.displayName || "Your Profile"}</Text>
      <Text style={styles.subtitle}>{user?.email}</Text>
      <Button title="Log Out" onPress={logOut} variant="secondary" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: "center", alignItems: "center", padding: 24,
  backgroundColor: "#fafafa" },
  title: { fontSize: 22, fontWeight: "800", color: "#111827", marginBottom: 4 },
  subtitle: { fontSize: 14, color: "#6b7280", marginBottom: 24 },
});
```

Why this changed: Removed the Day 4 placeholder text ("circles feed will appear here") since the real circles feed now exists as its own tab.

File 6 — AppNavigator.js MODIFIED

src/navigation/AppNavigator.js – replaces the Day 4 version

```
import React from "react";
import { createBottomTabNavigator } from "@react-navigation/bottom-tabs";
import { Ionicons } from "@expo/vector-icons";
import BrowseStackNavigator from "../BrowseStackNavigator";
import MyCirclesStackNavigator from "../MyCirclesStackNavigator";
import CreateCircleScreen from "../screens/CreateCircleScreen";
import HomeScreen from "../screens/HomeScreen";

const Tab = createBottomTabNavigator();

export default function AppNavigator() {
  return (
    <Tab.Navigator
      screenOptions={({ route }) => ({
        headerShown: false,
        tabBarActiveTintColor: "#2563eb",
        tabBarInactiveTintColor: "#9ca3af",
        tabBarIcon: ({ color, size }) => {
          let iconName = "ellipse";
          if (route.name === "BrowseTab") iconName = "compass-outline";
          else if (route.name === "MyCirclesTab") iconName = "people-outline";
          else if (route.name === "CreateTab") iconName = "add-circle-outline";
          else if (route.name === "ProfileTab") iconName = "person-outline";
          return <Ionicons name={iconName} size={size} color={color} />;
        },
      })
    >
      <Tab.Screen name="BrowseTab" component={BrowseStackNavigator} options={{ title:
"Browse" }} />
      <Tab.Screen name="MyCirclesTab" component={MyCirclesStackNavigator} options={{ title:
"My Circles" }} />
      <Tab.Screen name="CreateTab" component={CreateCircleScreen} options={{ title:
"Create" }} />
      <Tab.Screen name="ProfileTab" component={HomeScreen} options={{ title: "Profile" }} /
    >
    </Tab.Navigator>
  );
}
```

Where it goes: `src/navigation/AppNavigator.js` — replaces the Day 4 file. `RootNavigator.js` and `App.js` from Day 4 need no changes; they already render `AppNavigator` when a user is logged in.

Free tools confirmed for today: Firebase Firestore (Spark/free plan), React Navigation (open source), `@expo/vector-icons` (bundled free with Expo). No Anthropic API keys or any paid service is used anywhere in this app.

Run & Test

```
npx expo start
```

Scan the QR code with Expo Go (or press `a` / `i` for an emulator).

Full Day 5 Test Checklist

1. App opens to the **Browse** tab with 4 tabs at the bottom: Browse, My Circles, Create, Profile.
2. Tap **Create** → fill in a circle name + description → tap "Create Circle."
3. You're taken to that circle's **Detail** screen automatically, showing "1 member" and a "Leave Circle" button (since you auto-joined by creating it).
4. Go to **Browse** tab → your new circle appears in the list with a "Joined" badge.
5. Go to **My Circles** tab → the circle appears there too.
6. Tap the circle from Browse → tap **Leave Circle** → badge disappears, button changes to "Join Circle."
7. Tap **Join Circle** again → confirm it re-joins correctly.
8. **Profile** tab still shows your name/email and Log Out still works (from Day 4).

Commit & Push to GitHub

In Command Prompt, inside your project folder:

```
git add .  
git commit -m "Day 5: Add Circles feature (browse, create, join/leave) with bottom tab navigation"  
git push origin main
```

If your working branch is not named `main` (e.g. it's `master` or a feature branch), replace `main` with your actual branch name — check with `git branch` if unsure.

Day 5 Summary

Completed Today	Ready for Tomorrow
Firestore security rules published Circle data layer (create/read/join/leave) Browse Circles screen (live-updating list) Create Circle screen My Circles screen Circle Detail screen with Join/Leave Bottom tab navigation (4 tabs)	A working, navigable social app with real Firestore data Foundation ready for Posts inside each Circle (Day 6)

Day 6 objective: Posts feed inside each circle — users will be able to post updates, see a live feed per circle, and (per blueprint) start commenting.